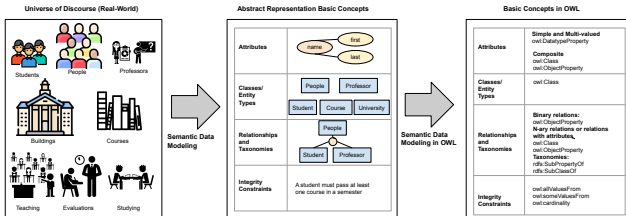


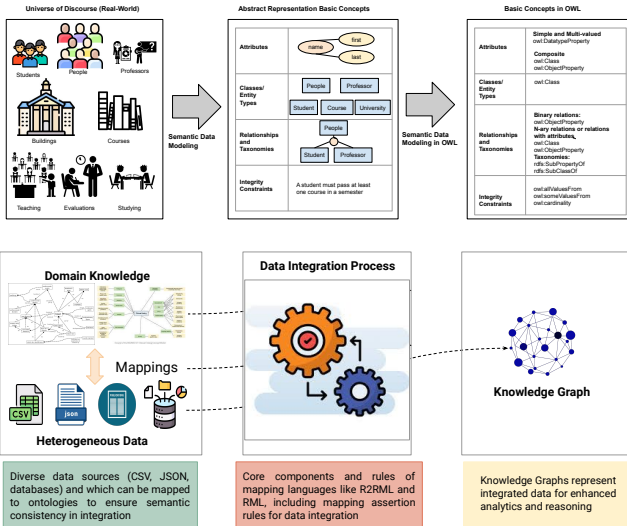
Knowledge Graph & Quality Assessment (Lecture 10)



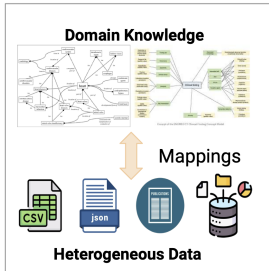
What have we learned so far?



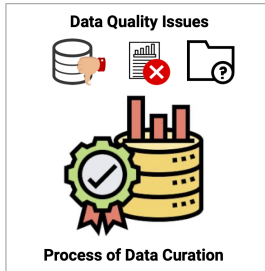
What have we learned so far?



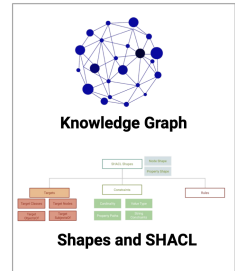
Knowledge Graph and Quality Assessment



Diverse data sources (CSV, JSON, databases) and which can be mapped to ontologies to ensure semantic consistency in integration

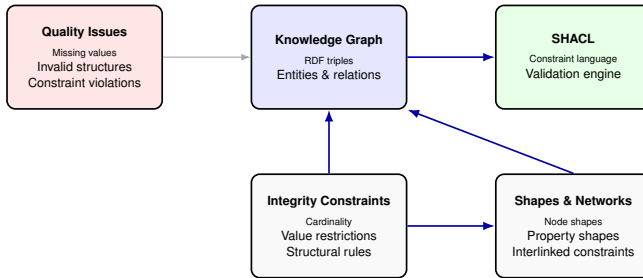


Data may suffer from inconsistencies, and missing and incorrect values. Data quality assessment and curation are needed to ensure that data and knowledge meet quality requirements



Shapes represent domain knowledge in the form of constraints. SHACL is the W3C recommendation language to specify shapes over KGs

Lecture 10 – Knowledge Graph Quality Assessment



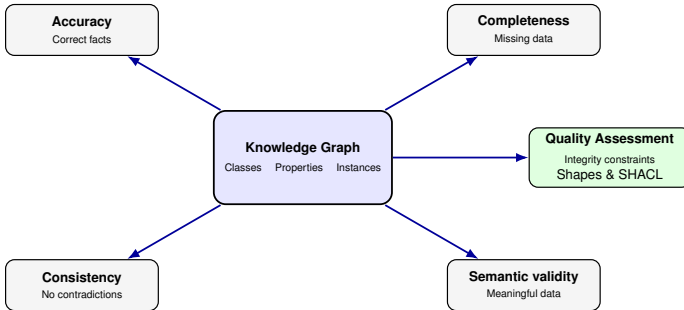
Lecture focus: Defining constraints and shapes, and validating Knowledge Graphs using **SHACL**.

Agenda — Lecture 10

- **Quality Assessment**
- **Integrity Constraints & Data Quality**
- **Shapes and SHACL**

Recommended reading: [1]

Quality Assessment — What We Will Study



Lecture focus: how to *formally assess and validate* these quality dimensions using constraints and SHACL.

Quality Assessment

Quality Assessment refers to

- evaluating data properties, processes, or outputs
- to determine their reliability, accuracy, and usefulness for intended applications.

Quality Assessment

Quality Assessment refers to

- evaluating data properties, processes, or outputs
- to determine their reliability, accuracy, and usefulness for intended applications.

For Knowledge Graphs (KGs), this includes

- assessing functional and non-functional requirements,
- ensuring data is accurate, complete, consistent, and semantically meaningful.

Problems of Quality in Knowledge Graphs

Functional Issues

- Incomplete or outdated data affects accuracy and topicality.
- Errors and inconsistencies lead to incorrect reasoning.
- Performance issues hinder accessibility.

Problems of Quality in Knowledge Graphs

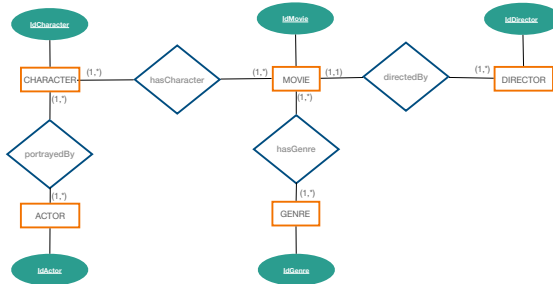
Functional Issues

- Incomplete or outdated data affects accuracy and topicality.
- Errors and inconsistencies lead to incorrect reasoning.
- Performance issues hinder accessibility.

Non-Functional Issues

- Lack of schema adherence or style guide compliance.
- Redundant and ambiguous data reduces usability.
- Insufficient documentation complicates maintenance and integration.

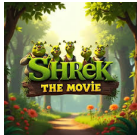
Quality Assessment Example (1/2)



Example: Cardinality Constraints

- Each movie must have exactly one director.
- Each movie must belong to at least one genre.
- Each movie must have at least one character.
- Each character must appear in at least one movie and be portrayed by at least one actor.

Quality Assessment Example (2/2)



```
@prefix ex: <http://example.org/> .  
@prefix rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#> .  
@prefix rdfs: <http://www.w3.org/2000/01/rdf-schema#> .
```

Movies

```
ex:Shrek a ex:Movie ;  
    ex:hasCharacter ex:ShrekCharacter,  
                    ex:Fiona,  
                    ex:Donkey ;  
ex:hasGenre ex:Comedy ;  
ex:hasReleaseYear "2001", "2002" ;  
ex:directedBy ex:AndrewAdamson .  
  
ex:ShrekForeverAfter a ex:Movie ;  
    ex:hasCharacter ex:Fiona, ex:Donkey ;  
ex:directedBy ex:MikeMitchell .
```

```
ex:Shrek2 a ex:Movie ;  
    ex:hasCharacter ex:Shrek,  
                    ex:Fiona,  
                    ex:Donkey ;  
ex:hasGenre ex:Comedy ;  
ex:hasReleaseYear "2004" ;  
ex:directedBy ex:AndrewAdamson,  
              ex:KellyAsbury,  
              ex:ConradVernon ;  
ex:director ex:AndrewAdamson .
```

Characters

```
ex:Shrek a ex:Character, ex:Ogre ;  
    ex:portrayedBy ex:MikeMyers .  
  
ex:Fiona a ex:Character,  
ex:HumanPrincess, ex:Ogre ;  
    ex:portrayedBy ex:CameronDiaz .  
  
ex:Donkey a ex:Character .
```

Data Quality Problems in Shrek KG (1/2)

Incompleteness

- The movie `ShrekForeverAfter` is missing a `hasGenre` relationship.
- The character `Donkey` is missing a `portrayedBy` relationship.

Inconsistencies

- The release year of `Shrek` is recorded as both "2001" and "2002".
- Multiple directors are listed for `Shrek2`, creating redundancy.

Data Quality Problems in Shrek KG (2/2)

Schema Violations

- Relationships to directors use inconsistent properties (`directedBy` and `director`).

Ambiguities

- The entity `Fiona` is represented as both `HumanPrincess` and `Ogre`, causing semantic confusion.

SPARQL Queries and Inconsistent Answers (1/2)

Query 1: Retrieve Movies and Their Release Years

```
PREFIX ex: <http://example.org/>
```

```
SELECT ?movie ?releaseYear
```

```
WHERE {
```

```
    ?movie ex:hasReleaseYear ?releaseYear .
```

```
}
```

```
# Query Output:
```

#	?movie	?releaseYear
#	ex:Shrek	"2001"
#	ex:Shrek	"2002"
#	ex:Shrek2	"2004"
#	ex:ShrekForeverAfter	Missing

SPARQL Queries and Inconsistent Answers (2/2)

Query 2: Retrieve Characters and Their Voice Actors

```
PREFIX ex: <http://example.org/>

SELECT ?character ?actor
WHERE {
    ?character ex:portrayedBy ?actor .
}
```

Query Output:

#	?character	?actor
#	ex:Shrek	ex:MikeMyers
#	ex:Fiona	ex:CameronDiaz
#	ex:Donkey	Missing

Reasons for Data Quality Issues (1/2)

Sources of Incompleteness

- **Data Entry Errors:** Missing relationships like `portrayedBy` for `Donkey` may result from incomplete data entry.
- **Lack of Metadata Standards:** No enforcement of rules to ensure every movie has a `hasGenre` relationship.

Reasons for Data Quality Issues (1/2)

Sources of Incompleteness

- **Data Entry Errors:** Missing relationships like `portrayedBy` for `Donkey` may result from incomplete data entry.
- **Lack of Metadata Standards:** No enforcement of rules to ensure every movie has a `hasGenre` relationship.

Causes of Inconsistencies

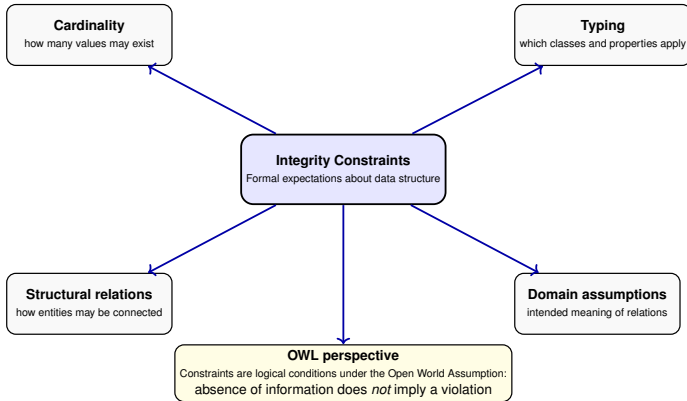
- **Conflicting Sources:** Multiple data providers with different release years for `Shrek`.
- **Manual Overrides:** Human edits introduced duplicate or conflicting values for the same property.

Reasons for Data Quality Issues (2/2)

Schema Violations and Ambiguities

- **Lack of Validation Mechanisms:** Relationships like `directedBy` and `director` coexist due to absent schema enforcement.
- **Insufficient Entity Disambiguation:** `Fiona` is ambiguously represented as both a `HumanPrincess` and an `Ogre`, lacking clear identification rules.

Integrity Constraints — What Do They Express?



Key idea: Integrity constraints describe how data *should be structured*, not whether data is complete.

Integrity Constraints with OWL (1/2)

Defining Cardinality Constraints in OWL

OWL can express constraints like:

- Each movie must have exactly one director:

```
ex:Movie a owl:Class ;  
  owl:equivalentClass [  
    a owl:Restriction ;  
    owl:onProperty ex:directedBy ;  
    owl:cardinality 1] .
```

- Each character must have at least one actor portraying it:

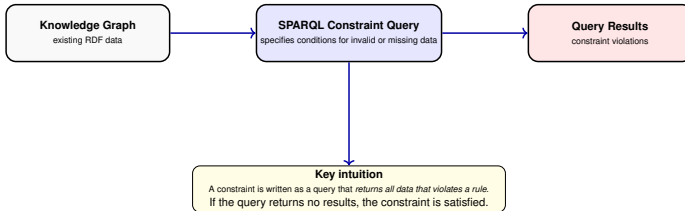
```
ex:Character a owl:Class ;  
  owl:equivalentClass [  
    a owl:Restriction ;  
    owl:onProperty ex:portrayedBy ;  
    owl:minCardinality 1] .
```

Integrity Constraints with OWL (2/2)

OWL Characteristics

- OWL follows the **Open World Assumption (OWA)**:
 - Missing data does not violate constraints.
 - Example: If no actors are listed for a character, OWL assumes they might exist elsewhere.

Integrity Constraints with SPARQL



Perspective: Constraint checking is performed by querying for violations.

Integrity Constraints with SPARQL Queries (1/4)

SPARQL Query for Cardinality Validation

```
PREFIX ex: <http://example.org/>

# Check if any movie has more than one director
SELECT ?movie
WHERE {
    ?movie a ex:Movie .
    OPTIONAL { ?movie ex:directedBy ?director1 . }
    OPTIONAL { ?movie ex:directedBy ?director2 . }
    FILTER (?director1 != ?director2)
}
```

Integrity Constraints with SPARQL Queries (2/4)

SPARQL Query for Cardinality Validation

```
PREFIX ex: <http://example.org/>

# Check if any movie has more than one director (Alternative)
SELECT ?movie
WHERE {
    ?movie a ex:Movie .
    ?movie ex:directedBy ?director . }
GROUP BY ?movie
HAVING (COUNT(?director) > 1)
}
```

Integrity Constraints with SPARQL Queries (3/4)

SPARQL Query for Cardinality Validation

```
# Check if any character is missing an actor portraying it
SELECT ?character
WHERE {
    ?character a ex:Character .
    FILTER NOT EXISTS { ?character ex:portrayedBy ?actor . }
}
```

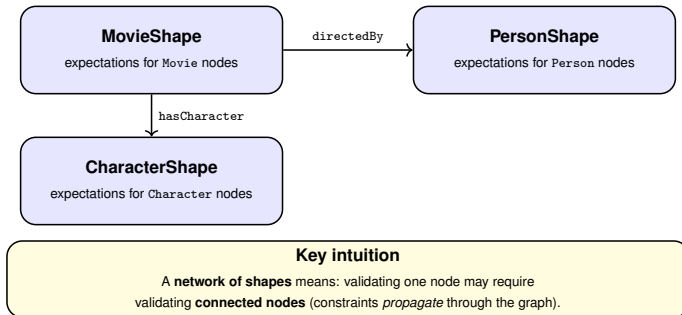
Integrity Constraints with SPARQL Queries (4/4)

Challenges with SPARQL

- Complex queries increase the risk of errors.
- Difficult to systematically validate all constraints.
- Manual validation of query results may be required.

Main limitation: SPARQL checks constraints by pattern matching, not by logic rules

Network of Shapes — Intuition



What is a Shape?

Definition of a Shape

- A shape is a standard logical expression to represent constraints in RDF knowledge graphs.
- It corresponds to a **conjunction of constraints** that a set of nodes in a knowledge graph must satisfy.
- Constraints are statements that impose restrictions on the values taken for target nodes in a given property.

Cardinality Constraints

- **Exactly N:** $[N..N]$
- **At least N:** $[N..*]$
- **At most N:** $[0..N]$

Definition of a Shape

Definition of a Shape

- A shape is graphically represented as a rectangle.
- Each rectangle includes a target node (e.g., a class `Movie`, `Character`).
- Inside the rectangle:
 - Properties (object and data type properties) are listed, along with their data types and constraints (e.g., `ex:title: xsd:string [1..1]`).

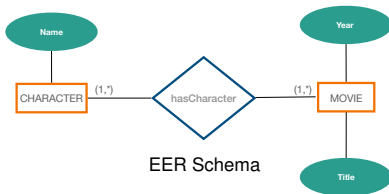
Networks of Shapes

- Shapes are connected to represent relationships between classes.
- Edges between shapes specify relationships and their constraints (e.g., `hasCharacter: [1..*]`).

Networks of Shapes (1/2)

Representation of the Movie and Character Shapes

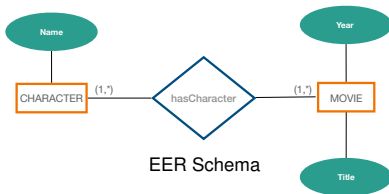
- A Movie must:
 - Have a title (`xsd:string [1..1]`).
 - Have a year (`xsd:date [1..1]`).
 - Have at least one Character (`hasCharacter: [1..*]`).
- A Character must:
 - Have a name (`xsd:string [1..1]`).
 - Be part of at least one Movie (`isCharacterIn: [1..*]`).



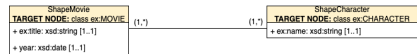
Networks of Shapes (1/2)

Representation of the Movie and Character Shapes

- A Movie must:
 - Have a title (`xsd:string [1..1]`).
 - Have a year (`xsd:date [1..1]`).
 - Have at least one Character (`hasCharacter: [1..*]`).
- A Character must:
 - Have a name (`xsd:string [1..1]`).
 - Be part of at least one Movie (`isCharacterIn: [1..*]`).



EER Schema



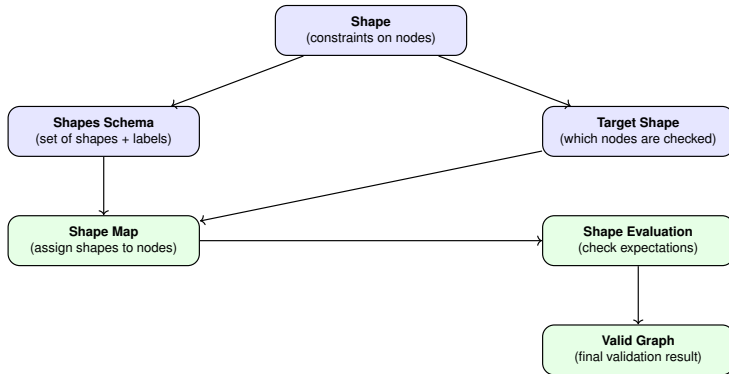
Shape Network

Networks of Shapes (2/2)

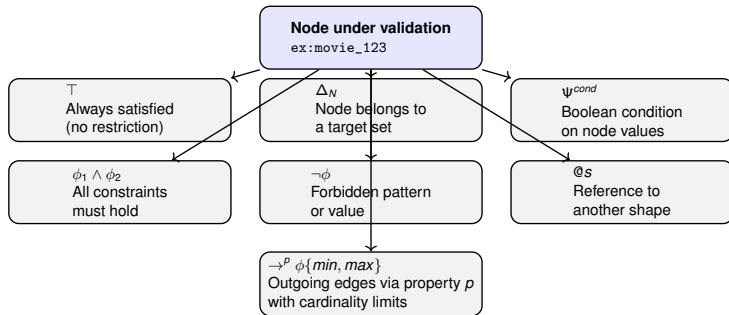
Key Constraints in the Shrek Knowledge Graph

- `Movie` and `Character` are interdependent:
 - A `Movie` must include at least one `Character` (`hasCharacter`: [1..*]).
 - A `Character` must appear in at least one `Movie` (`isCharacterIn`: [1..*]).
- Properties like `title` and `name` ensure mandatory attributes with specific data types.

Formalization of Shapes



Shapes — Intuition Formal Definition



A shape is built by combining these components into a logical specification that determines whether a node is valid.

Formal Definition of a Shape

Definition 1

A shape ϕ is defined as:

$\phi ::= \top$ (true)

| Δ_N (node belongs to the set of nodes N)

| ψ^{cond} (node satisfies the Boolean condition cond)

| $\phi_1 \wedge \phi_2$ (conjunction of shape ϕ_1 and shape ϕ_2)

| $\neg\phi$ (negation of shape ϕ)

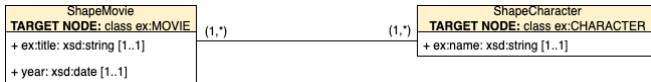
| $@s$ (reference to shape with label s)

| $\rightarrow^p \phi \{\text{min}, \text{max}\}$ (between min and max outward edges)
with label p to nodes satisfying ϕ .

Notes

- $\text{min} \in \mathbb{N}$, $\text{max} \in \mathbb{N} \cup \{*\}$, where $*$ indicates unbounded.

Shape Example: Movie and Character



ex:ShapeMovie : TARGET CLASS: ex:Movie

→ ex:title Δ xsd:string {1,1}

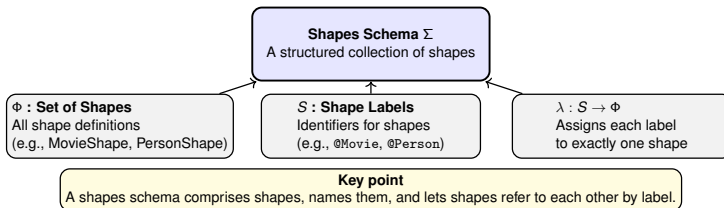
→ ex:year Δ xsd:date {1,1}

→^{hasCharacter} exS:Character {1,*}

ex:ShapeCharacter : TARGET CLASS: ex:Character

→ ex:name Δ xsd:string {1,1}

Shapes Schema — Intuition



Shapes Schema Definition

Definition 2

A shapes schema is defined as a tuple $\Sigma = (\Phi, S, \lambda)$, where:

- Φ is a set of shapes.
- S is a set of shape labels.
- $\lambda : S \rightarrow \Phi$ is a total function from labels to shapes.

Shapes Schema: Movies and Characters (1/2)

Shapes Schema

- $\Sigma = (\Phi, S, \lambda)$
- Φ : Set of shapes
- S : Set of shape labels
- λ : Total function mapping labels to shapes

Shapes Schema: Movies and Characters (2/2)

Components of Σ

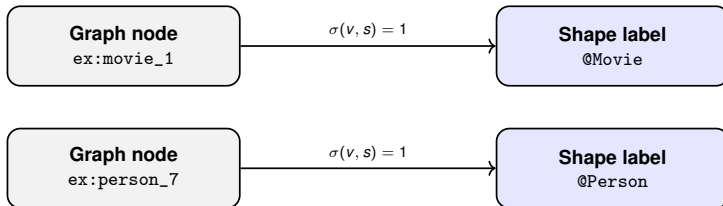
- $\Phi = \{$
 - $\rightarrow \text{ex:title } \Delta\text{xsd:string } \{1,1\},$
 - $\rightarrow \text{ex:year } \Delta\text{xsd:date } \{1,1\},$
 - $\rightarrow^{\text{hasCharacter}} \text{ex:ShapeCharacter } \{1,*\},$
 - $\rightarrow \text{ex:name } \Delta\text{xsd:string } \{1,1\}\}$
- $S = \{\text{ex:ShapeMovie}, \text{ex:ShapeCharacter}\}$
- $\lambda(\text{ex:ShapeMovie}) =$
 - $\rightarrow \text{ex:title } \Delta\text{xsd:string } \{1,1\}$
 - $\wedge \rightarrow \text{ex:year } \Delta\text{xsd:date } \{1,1\}$
 - $\wedge \rightarrow^{\text{hasCharacter}} \text{ex:ShapeCharacter } \{1,*\}$
- $\lambda(\text{ex:ShapeCharacter}) =$
 - $\rightarrow \text{ex:name } \Delta\text{xsd:string } \{1,1\}$

Shapes Map — Intuition

Shapes map $\sigma(v, s)$ specifies *which nodes (v) are expected to satisfy which shapes (s)*.

Shapes Map — Intuition

Shapes map $\sigma(v, s)$ specifies *which nodes (v) are expected to satisfy which shapes (s)*.



Shapes Map

Definition 3

Given a directed edge-labeled graph $G = (V, E, L)$ and a shapes schema $\Sigma = (\Phi, S, \lambda)$, a shapes map is a (partial) mapping:

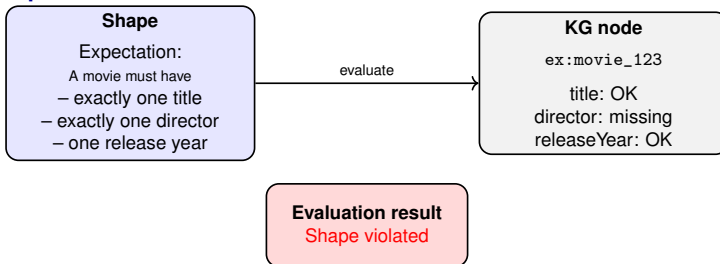
$$\sigma : V \times S \rightarrow \{0, 1\}$$

Explanation

The shapes map σ is a way of labeling the nodes of G with the labels of shapes in S :

- $\sigma(v, s) = 1$, then node v is labeled with s
- Otherwise, $\sigma(v, s) = 0$

Shape Evaluation — Intuition



Key intuition
Shape evaluation checks whether a node *meets all expectations*.
The result is **pass** (1) or **violation** (0).

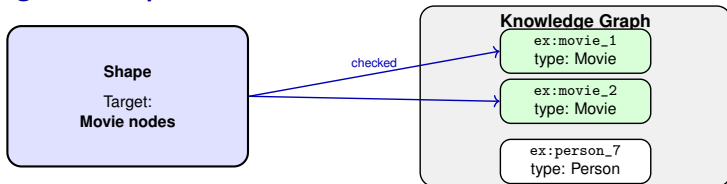
Shape Evaluation

Definition 4

Given a shapes schema $\Sigma = (\Phi, S, \lambda)$ and a directed edge-labeled graph $G = (V, E, L)$, a node $v \in V$, and a total shapes map σ , the shape evaluation function $[\phi]^{G,v,\sigma} \in \{0, 1\}$ is defined as follows:

- $[\top]^{G,v,\sigma} = 1$
- $[\Delta_N]^{G,v,\sigma} = 1 \iff v \in N$
- $[\Psi^{\text{cond}}]^{G,v,\sigma} = 1 \iff \text{cond}(v) \text{ is true}$
- $[\phi_1 \wedge \phi_2]^{G,v,\sigma} = \min\{[\phi_1]^{G,v,\sigma}, [\phi_2]^{G,v,\sigma}\}$
- $[\neg\phi]^{G,v,\sigma} = 1 - [\phi]^{G,v,\sigma}$
- $[@s]^{G,v,\sigma} = 1 \iff \sigma(v, s) = 1$
- $[\rightarrow^p \phi\{\text{min}, \text{max}\}]^{G,v,\sigma} = 1$
 $\iff$
 $\min \leq |\{(v, p, u) \in E \mid [\phi]^{G,u,\sigma} = 1\}| \leq \max$
- If $[\phi]^{G,v,\sigma} = 1$, then v is said to satisfy ϕ in G under σ .

Target Shape – Intuition



A target shape specifies *which nodes are checked*. Only nodes matching the target are evaluated against the shape.

Shapes Target

Definition 5

Given a shapes schema $\Sigma = (\Phi, S, \lambda)$ and a directed edge-labeled graph $G = (V, E, L)$:

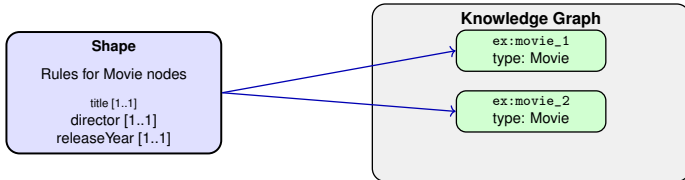
- A shapes target $T \subseteq V \times S$ is a set of pairs of nodes and shape labels from G and Σ , respectively.

Example

In our running example:

$$T = \{(ex:Author, exS:Author), (ex:Paper, exS:Paper)\}$$

Valid Graph — Intuition



Key intuition

A graph is **valid** if

- every *targeted node* is checked, and
- **all checks succeed.**

No required node violates its assigned shape.

Valid Graph

Definition 6

Given a shapes schema $\Sigma = (\Phi, S, \lambda)$, a directed edge-labeled graph $G = (V, E, L)$, and a shapes target T :

- G is valid under Σ and T iff:
 - There exists a shapes map σ , such that:
 - For all $s \in S$ and $v \in V$, it holds that:

$$\sigma(v, s) = [\lambda(s)]^{G, v, \sigma}$$

- $(v, s) \in T$ implies $\sigma(v, s) = 1$

Example Validation- Same Graph from Slide 10 (1/3)

```
@prefix ex: <http://example.org/> .  
# Movies  
ex:Shrek a ex:Movie ;  
    ex:hasCharacter ex:ShrekCharacter, ex:Fiona, ex:Donkey ;  
    ex:hasGenre ex:Comedy ;  
    ex:hasReleaseYear "2001", "2002" ;  
    ex:directedBy ex:AndrewAdamson .  
ex:ShrekForeverAfter a ex:Movie ;  
    ex:hasCharacter ex:Fiona, ex:Donkey ;  
    ex:directedBy ex:MikeMitchell .  
# Characters  
ex:Donkey a ex:Character .  
ex:Fiona a ex:Character, ex:HumanPrincess, ex:Ogre .
```

Example Validation- Same Graph from Slide 10 (1/3)

```
@prefix ex: <http://example.org/> .  
# Movies  
ex:Shrek a ex:Movie ;  
    ex:hasCharacter ex:ShrekCharacter, ex:Fiona, ex:Donkey ;  
    ex:hasGenre ex:Comedy ;  
    ex:hasReleaseYear "2001", "2002" ;  
    ex:directedBy ex:AndrewAdamson .  
ex:ShrekForeverAfter a ex:Movie ;  
    ex:hasCharacter ex:Fiona, ex:Donkey ;  
    ex:directedBy ex:MikeMitchell .  
# Characters  
ex:Donkey a ex:Character .  
ex:Fiona a ex:Character, ex:HumanPrincess, ex:Ogre .
```

Validation

- Target nodes (T):

$$T = \{(ex:Shrek, ex:ShapeMovie), (ex:ShrekForeverAfter, ex:ShapeMovie), \\ (ex:Donkey, ex:ShapeCharacter), (ex:Fiona, ex:ShapeCharacter)\}$$

- All nodes in T satisfy their shapes under the schema.
- The graph is valid under Σ and T .

Example Validation (2/3)

Validation of Nodes Against Shapes

- **Movie:**
 - 'ex:Shrek' satisfies 'ex:ShapeMovie':
 - 'ex:hasCharacter' references valid 'ex:ShapeCharacter' nodes.
 - 'ex:hasReleaseYear' contains valid 'xsd:date' values.
 - 'ex:ShrekForeverAfter' satisfies 'ex:ShapeMovie':
 - 'ex:hasCharacter' references valid 'ex:ShapeCharacter' nodes.

Example Validation (3/3)

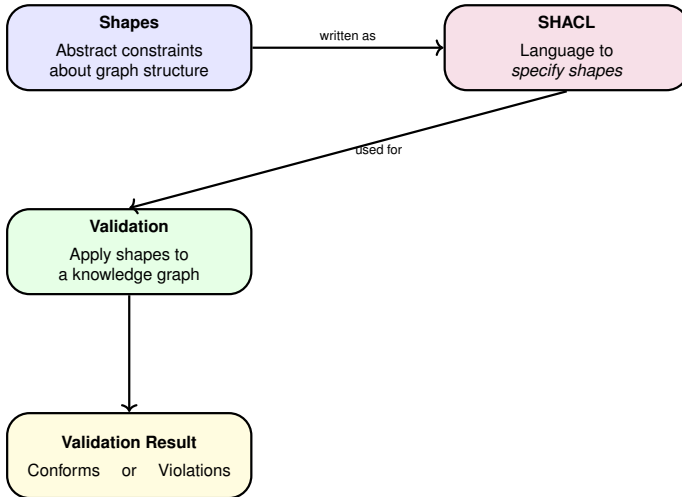
Validation of Nodes Against Shapes

- Character:
 - 'ex:Donkey' satisfies 'ex:ShapeCharacter'.
 - 'ex:Fiona' satisfies 'ex:ShapeCharacter'.

Conclusion

The graph is valid under Σ and T , as all target nodes satisfy their respective shapes.

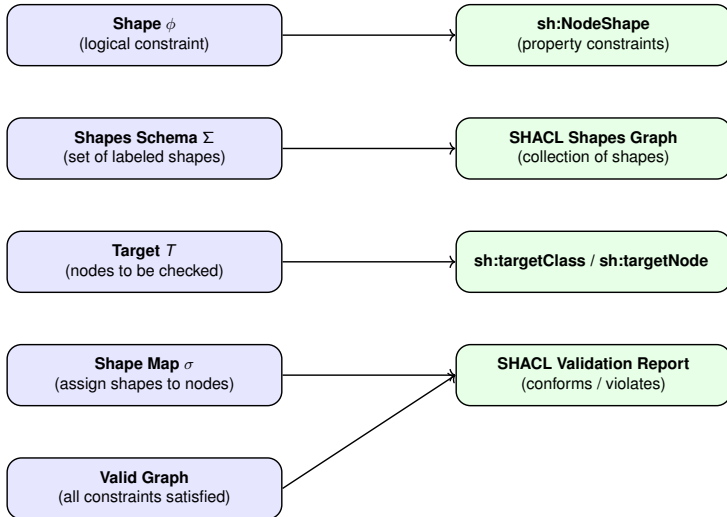
SHACL — Specifying Shapes



What is SHACL?

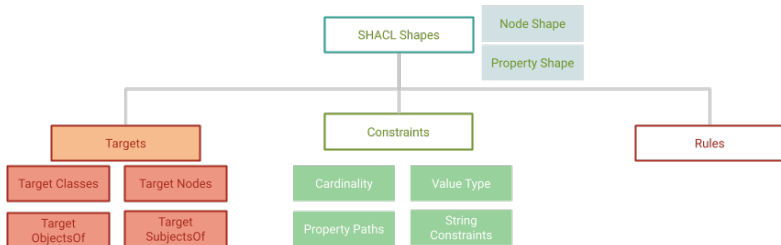
- SHACL: Shapes Constraint Language.
- W3C recommendation language for specifying integrity constraints over RDF Knowledge Graphs.
- SHACL shapes:
 - Represent a set of constraints applied to RDF entities.
 - Can reference other shapes to define inter-entity constraints.
- More details: <https://www.w3.org/TR/shacl/>

From Formal Shapes to SHACL



Defining a Shape in SHACL

- A shape in SHACL:
 - Represents a collection of constraints applied to RDF data.
 - Targets specific RDF nodes using `sh:targetClass`, `sh:targetNode`, or `sh:targetObjectsOf`.
 - Ensures that RDF data conforms to expected structures and rules.
- Shapes can:
 - Validate node-level constraints (Node Shapes).
 - Validate property-level constraints (Property Shapes).



SHACL Operators — At a Glance

Targeting

sh:targetClass
sh:targetNode
sh:targetSubjectsOf
sh:targetObjectsOf

Property Constraints

sh:property
sh:path
sh:minCount / sh:maxCount
sh:datatype

Severity

sh:severity
Violation / Warning

Logical Operators

sh:and
sh:or
sh:not
sh:xone

Paths

direct
inverse
sequence
alternative
zero-or-more

Advanced Constraints

sh:closed
sh:in / sh:hasValue
sh:sparql

Key idea: SHACL operators answer *six questions* — *who, what, how many, how related, how combined, and how strict*.

Main Operators in a SHACL Shape

■ Targeting:

- `sh:targetClass`: Specifies the RDF class the shape applies to.
- `sh:targetNode`: Targets specific nodes in the RDF graph.
- `sh:targetObjectsOf`: Targets objects of a specific property.

■ Constraints:

- `sh:property`: Defines property-level constraints.
- `sh:path`: Specifies the property path to validate.
- Cardinality Constraints: `sh:minCount`, `sh:maxCount`.
- Datatype Constraints: `sh:datatype`.

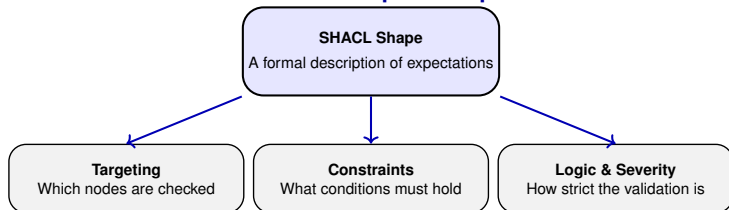
■ Severity:

- `sh:severity`: Defines the level of severity (e.g., `sh:Violation`, `sh:Warning`).

■ Logical Operators:

- `sh:and`, `sh:or`, `sh:not`, `sh:xone`.

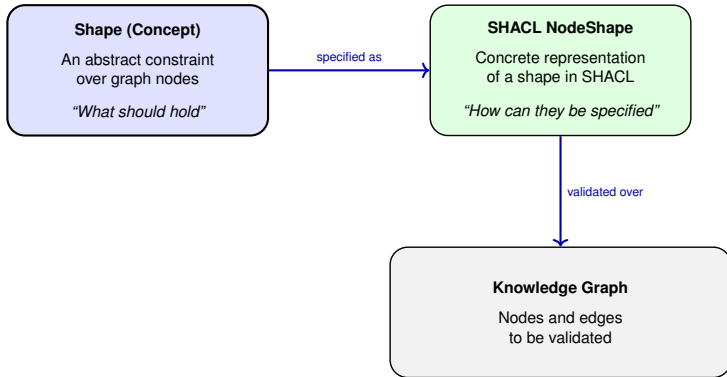
What Can a SHACL Shape Express?



Intuition:

A SHACL shape combines *who to validate*, *what to check*, and *how to interpret violations*.

What Is a SHACL Node Shape?



Key intuition:

A **NodeShape** is the SHACL construct used to specify a shape.
It is the *formal encoding* of an abstract constraint.

SHACL Node Shapes

- Node Shape example for MOVIES:

```
@prefix ex: <http://example.org/> .  
@prefix exS: <http://example.org/shapes> .  
@prefix sh: <http://www.w3.org/ns/shacl#> .  
@prefix xsd: <http://www.w3.org/2001/XMLSchema#> .
```

```
exS:Movie a sh:NodeShape ;  
  sh:targetClass ex:Movie ;  
  sh:property [  
    sh:path ex:title ;  
    sh:datatype xsd:string ;  
    sh:minCount 1 ;  
    sh:maxCount 1],  
  [ sh:path ex:releaseYear ;  
    sh:datatype xsd:Year ;  
    sh:minCount 1].
```


SHACL Node Shapes

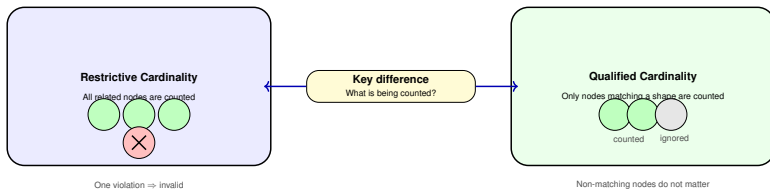
- Node Shape example for MOVIES:

```
@prefix ex: <http://example.org/> .  
@prefix exS: <http://example.org/shapes> .  
@prefix sh: <http://www.w3.org/ns/shacl#> .  
@prefix xsd: <http://www.w3.org/2001/XMLSchema#> .
```

```
exS:Movie a sh:NodeShape ;  
  sh:targetClass ex:Movie ;  
  sh:property [  
    sh:path ex:title ;  
    sh:datatype xsd:string ;  
    sh:minCount 1 ;  
    sh:maxCount 1],  
  [ sh:path ex:releaseYear ;  
    sh:datatype xsd:Year ;  
    sh:minCount 1].
```

- Key operators:
 - `sh:targetClass`: Specifies the target RDF class (e.g., `Movie`).
 - `sh:property`: Defines constraints on properties (e.g., `title`, `release year`).
 - `sh:minCount`, `sh:maxCount`: Cardinality constraints.

Types of Cardinality Constraints — An Intuition



Key intuition: *Restrictive counts everything — Qualified counts only what matches a shape.*

Restrictive Cardinality Constraints

Given:

- A directed edge-labeled graph $G = (V, E, L)$.
- Set N of target nodes in a shape S .
- A path p in S .
- V_n^p : Nodes in V connected by path p from each target node $n \in N$.

Restrictive Constraint C :

- Imposes a cardinality constraint over path p .
- Ensures that **all nodes** in V_n^p must satisfy the shape S .

Operators in SHACL:

- `sh:maxCount m`: At most m nodes in V_n^p .
- `sh:minCount m`: At least m nodes in V_n^p .

Example-Restrictive Constraints

RDF Graph:

```
ex:Movie1 a ex:Movie ;
  ex:title "Inception" ;
  ex:hasDirector ex:Director1 .

ex:Director1 a ex:Director ;
  ex:name "Christopher Nolan" .

ex:Movie2 a ex:Movie ;
  ex:title "Untitled" ;
  ex:hasDirector ex:Director2 .

ex:Director2 a ex:Director .
```

SHACL Shapes:

```
exS:Director a sh:NodeShape ;
  sh:targetClass ex:Director ;
  sh:property [
    sh:path ex:name ;
    sh:datatype xsd:string ;
    sh:minCount 1 ;
    sh:maxCount 1] .

exS:Movie a sh:NodeShape ;
  sh:targetClass ex:Movie ;
  sh:property [
    sh:path ex:title ;
    sh:datatype xsd:string ;
    sh:minCount 1 ;
    sh:maxCount 1],
  [ sh:path ex:hasDirector ;
    sh:node exS:Director ;
    sh:minCount 1] .
```

Example-Restrictive Constraints

RDF Graph:

```
ex:Movie1 a ex:Movie ;  
  ex:title "Inception" ;  
  ex:hasDirector ex:Director1 .  
  
ex:Director1 a ex:Director ;  
  ex:name "Christopher Nolan" .  
  
ex:Movie2 a ex:Movie ;  
  ex:title "Untitled" ;  
  ex:hasDirector ex:Director2 .  
  
ex:Director2 a ex:Director .
```

SHACL Shapes:

```
exS:Director a sh:NodeShape ;  
  sh:targetClass ex:Director ;  
  sh:property [  
    sh:path ex:name ;  
    sh:datatype xsd:string ;  
    sh:minCount 1 ;  
    sh:maxCount 1] .  
  
exS:Movie a sh:NodeShape ;  
  sh:targetClass ex:Movie ;  
  sh:property [  
    sh:path ex:title ;  
    sh:datatype xsd:string ;  
    sh:minCount 1 ;  
    sh:maxCount 1] ,  
  [ sh:path ex:hasDirector ;  
    sh:node exS:Director ;  
    sh:minCount 1] .
```

- Movie1 validates because:
 - It has exactly one title (Inception).
 - It has a valid Director (Director1) with a name.
- Movie2 does not validate because:
 - The Director (Director2) lacks a name.

Qualified Cardinality Constraints

Given:

- A directed edge-labeled graph $G = (V, E, L)$.
- Set N of target nodes in a shape S .
- A path p in S .
- V_n^p : Nodes in V connected by path p from each target node $n \in N$.
- M : Class in the range of p .
- T : A shape that targets entities in M .

Qualified Constraint C :

- Imposes that the number of nodes in V_n^p must satisfy T .
- There can be nodes in V_n^p that do not satisfy the shape T .

Operators in SHACL:

- `sh:qualifiedMaxCount m T`: At most m nodes in V_n^p satisfy T .
- `sh:qualifiedMinCount m T`: At least m nodes in V_n^p satisfy T .

Example-Qualified Constraints

Example RDF Graph:

```
ex:Movie1 a ex:Movie ;
  ex:title "Inception" ;
  ex:hasActor ex:Actor1, ex:Actor2 .

ex:Actor1 a ex:Actor ;
  ex:name "Leonardo DiCaprio" .

ex:Actor2 a ex:Actor ;
  ex:name "Joseph Gordon-Levitt" .

ex:Movie2 a ex:Movie ;
  ex:title "Dunkirk" ;
  ex:hasActor ex:Actor3 .

ex:Actor3 a ex:Actor .
```

SHACL Shapes:

```
exS:Movie a sh:NodeShape ;
  sh:targetClass ex:Movie ;
  sh:property [
    sh:path ex:hasActor ;
    sh:qualifiedValueShape exS:Actor ;
    sh:qualifiedMinCount 1 ;
    sh:qualifiedMaxCount 3
  ].

exS:Actor a sh:NodeShape ;
  sh:targetClass ex:Actor ;
  sh:property [
    sh:path ex:name ;
    sh:datatype xsd:string ;
    sh:minCount 1 ].
```

Example-Qualified Constraints

Example RDF Graph:

```
ex:Movie1 a ex:Movie ;  
  ex:title "Inception" ;  
  ex:hasActor ex:Actor1, ex:Actor2 .
```

```
ex:Actor1 a ex:Actor ;  
  ex:name "Leonardo DiCaprio" .
```

```
ex:Actor2 a ex:Actor ;  
  ex:name "Joseph Gordon-Levitt" .
```

```
ex:Movie2 a ex:Movie ;  
  ex:title "Dunkirk" ;  
  ex:hasActor ex:Actor3 .
```

```
ex:Actor3 a ex:Actor .
```

SHACL Shapes:

```
exS:Movie a sh:NodeShape ;  
  sh:targetClass ex:Movie ;  
  sh:property [  
    sh:path ex:hasActor ;  
    sh:qualifiedValueShape exS:Actor ;  
    sh:qualifiedMinCount 1 ;  
    sh:qualifiedMaxCount 3  
  ] .
```

```
exS:Actor a sh:NodeShape ;  
  sh:targetClass ex:Actor ;  
  sh:property [  
    sh:path ex:name ;  
    sh:datatype xsd:string ;  
    sh:minCount 1 ] .
```

- Movie1 validates because:
 - It has exactly two Actors (Actor1 and Actor2).
 - Both Actor1 and Actor2 have valid name properties.
- Movie2 does not validate because:
 - It has only one Actor (Actor3).
 - Actor3 lacks a name property.

SHACL Target Nodes — An Intuition

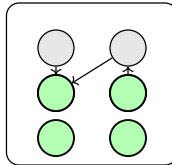
Target by Class

All nodes of a given type

Target by Subject

Nodes that appear as subjects of a property

Knowledge Graph



Target by Object

Nodes that appear as objects of a property

Target Specific Node

Exactly one chosen node

Key intuition

Targeting decides *which nodes are checked*. Constraints are evaluated *only* on the selected nodes.

SHACL: Defining Target Nodes

- SHACL (Shapes Constraint Language) supports flexible targeting mechanisms for node validation:
 - `sh:targetClass C`: Targets all nodes belonging to class *C*.
 - `sh:targetSubjectsOf P`: Targets entities that are the subjects of triples with property *P*.
 - `sh:targetObjectsOf P`: Targets entities that are the objects of triples with property *P*.
 - `sh:targetNode e`: Targets a specific node *e*.
- These targets help enforce integrity constraints on RDF data.

Example: sh:targetSubjectsOf

RDF Graph:

```
ex:Movie1 a ex:MOVIE ;
    ex:title "Inception" ;
    ex:hasCharacter ex:Character1 .
ex:Movie2 a ex:MOVIE ;
    ex:title "The Matrix" ;
    ex:hasCharacter ex:Character2 .
ex:Character1 a ex:CHARACTER ;
    ex:name "Dom Cobb" .
ex:Character2 a ex:CHARACTER ;
    ex:name "Neo" .
```

SHACL Shapes:

```
ex:MovieShape a sh:NodeShape ;
    sh:targetSubjectsOf ex:hasCharacter ;
    sh:property [
        sh:path ex:title ;
        sh:datatype xsd:string ] .
```

Evaluation Results:

- ex:Movie1: Valid (Title is a string)
- ex:Movie2: Valid (Title is a string)
- ex:Character1 and ex:Character2 are not evaluated as they are not subjects of ex:hasCharacter.

Example: sh:targetObjectsOf

RDF Graph:

```
ex:Movie1 a ex:MOVIE ;
    ex:title "Inception" ;
    ex:hasCharacter ex:Character1 .
ex:Movie2 a ex:MOVIE ;
    ex:title "The Matrix" ;
    ex:hasCharacter ex:Character2 .
ex:Character1 a ex:CHARACTER ;
    ex:name "Dom Cobb" .
ex:Character2 a ex:CHARACTER ;
    ex:name "Neo" .
```

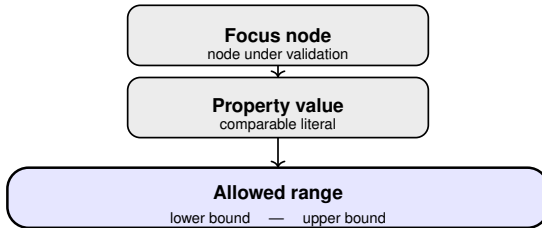
SHACL Shapes:

```
ex:CharacterShape a sh:NodeShape ;
    sh:targetObjectsOf ex:hasCharacter ;
    sh:property [
        sh:path ex:name ;
        sh:datatype xsd:string ] .
```

Evaluation Results:

- ex:Character1: Valid (Name is a string)
- ex:Character2: Valid (Name is a string)
- ex:Movie1 and ex:Movie2 are not evaluated as they are not objects of ex:hasCharacter.

SHACL Value Range Constraints – Intuition



inside range



outside range



Key intuition
Value range constraints check whether values fall inside permitted bounds

SHACL: Value Range Constraints

- SHACL allows specifying value range constraints for target nodes.
- These constraints ensure numerical or comparable properties of nodes fall within defined ranges.
- `sh:minInclusive m`: Ensures the value of n satisfies $n \geq m$.
- `sh:maxInclusive m`: Ensures the value of n satisfies $n \leq m$.
- `sh:minExclusive m`: Ensures the value of n satisfies $n > m$.
- `sh:maxExclusive m`: Ensures the value of n satisfies $n < m$.

SHACL: Value Range Constraints

- SHACL allows specifying value range constraints for target nodes.
- These constraints ensure numerical or comparable properties of nodes fall within defined ranges.
- `sh:minInclusive m`: Ensures the value of n satisfies $n \geq m$.
- `sh:maxInclusive m`: Ensures the value of n satisfies $n \leq m$.
- `sh:minExclusive m`: Ensures the value of n satisfies $n > m$.
- `sh:maxExclusive m`: Ensures the value of n satisfies $n < m$.

Use Case: Enforcing numerical bounds for attributes like age, price, or scores.

Example: SHACL Value Range Constraints

RDF Graph:

```
ex:Movie1 a ex:MOVIE ;
    ex:title "Inception" ;
    ex:rating 8.8 .

ex:Movie2 a ex:MOVIE ;
    ex:title "The Matrix" ;
    ex:rating 8.7 .

ex:Movie3 a ex:MOVIE ;
    ex:title "Generic Movie" ;
    ex:rating 4.3 .
```

SHACL Shapes:

```
ex:MovieShape a sh:NodeShape ;
    sh:targetClass ex:MOVIE ;
    sh:property [
        sh:path ex:rating ;
        sh:datatype xsd:decimal ;
        sh:minInclusive 5.0 ;
        sh:maxInclusive 10.0 ] .
```


Example: SHACL Value Range Constraints

RDF Graph:

```
ex:Movie1 a ex:MOVIE ;  
    ex:title "Inception" ;  
    ex:rating 8.8 .  
  
ex:Movie2 a ex:MOVIE ;  
    ex:title "The Matrix" ;  
    ex:rating 8.7 .  
  
ex:Movie3 a ex:MOVIE ;  
    ex:title "Generic Movie" ;  
    ex:rating 4.3 .
```

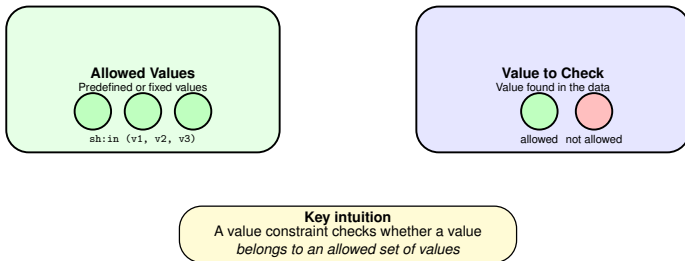
SHACL Shapes:

```
ex:MovieShape a sh:NodeShape ;  
    sh:targetClass ex:MOVIE ;  
    sh:property [  
        sh:path ex:rating ;  
        sh:datatype xsd:decimal ;  
        sh:minInclusive 5.0 ;  
        sh:maxInclusive 10.0 ] .
```

Evaluation Results:

- ex:Movie1: Valid (8.8 satisfies $5.0 \leq n \leq 10.0$).
- ex:Movie2: Valid (8.7 satisfies $5.0 \leq n \leq 10.0$).
- ex:Movie3: Invalid (4.3 does not satisfy $5.0 \leq n \leq 10.0$).

SHACL Value Constraints — An Intuition



Interpretation:

sh:hasValue checks equality — sh:in checks membership.

SHACL: Value Constraints

- SHACL allows for specifying exact or enumerated values for target nodes:
 - `sh:hasValue v`: Ensures that the value v belongs to the set of target nodes N .
 - `sh:in (v1...vn)`: Ensures that the values in the set of target nodes N are a subset of $\{v1, ..., vn\}$.
- These constraints enforce specific or predefined values for RDF properties.

SHACL: Value Constraints

- SHACL allows for specifying exact or enumerated values for target nodes:
 - `sh:hasValue v`: Ensures that the value v belongs to the set of target nodes N .
 - `sh:in (v1...vn)`: Ensures that the values in the set of target nodes N are a subset of $\{v1, ..., vn\}$.
- These constraints enforce specific or predefined values for RDF properties.

Use Case: Ensuring specific property values for controlled vocabularies, predefined attributes, or categories.

Example: SHACL Value Constraints

RDF Graph:

```
ex:Movie1 a ex:MOVIE ;  
    ex:title "Inception" ;  
    ex:genre "Sci-Fi" .  
  
ex:Movie2 a ex:MOVIE ;  
    ex:title "The Matrix" ;  
    ex:genre "Action" .  
  
ex:Movie3 a ex:MOVIE ;  
    ex:title "Generic Movie" ;  
    ex:genre "Unknown" .
```

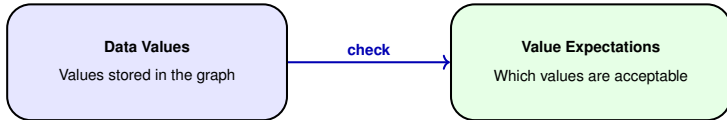
SHACL Shapes:

```
ex:MovieShape a sh:NodeShape ;  
    sh:targetClass ex:MOVIE ;  
    sh:property [  
        sh:path ex:genre ;  
        sh:in ("Sci-Fi" "Action" "Drama")  
    ] .
```

Evaluation Results:

- ex:Movie1: Valid ("Sci-Fi" is in the allowed values).
- ex:Movie2: Valid ("Action" is in the allowed values).
- ex:Movie3: Invalid ("Unknown" is not in {"Sci-Fi", "Action", "Drama"}).

SHACL Value Constraints — Intuition



Key idea

Value constraints verify whether *values in the data* match *expected or allowed values*.

SHACL: SPARQL Constraints

- SHACL allows defining constraints using SPARQL queries, enabling complex validation rules.
- SPARQL-based constraints are of type `sh:SPARQLConstraint` and include:
 - `sh:message`: Provides a human-readable explanation for the violation.
 - `sh:select`: Declares the SPARQL query.
 - The query can use the special variable `$this`, which refers to the target node.
 - Nodes returned by the query indicate violations of the constraint.
- SPARQL-based constraints allow validating relationships and values across multiple nodes and properties.

Example: SPARQL Constraint (1/2)

RDF Graph:

```
ex:Movie1 a ex:MOVIE ;
    ex:title "Inception" ;
    ex:rating 8.8 ;
    ex:year "2010" .

ex:Movie2 a ex:MOVIE ;
    ex:title "The Matrix" ;
    ex:rating 9.5 ;
    ex:year "1999" .

ex:Movie3 a ex:MOVIE ;
    ex:title "Generic Movie" ;
    ex:rating 12.0 ;
    ex:year "2023" .
```

SHACL Shapes:

```
ex:MovieShape a sh:NodeShape ;
    sh:targetClass ex:MOVIE ;
    sh:sparql [
        a sh:SPARQLConstraint ;
        sh:message "Rating must be between 0 and 10" ;
        sh:select ""
            SELECT $this
            WHERE {$this ex:rating ?rating .
                FILTER (?rating < 0 || ?rating > 10)
            }"" ] .
```

Evaluation Results:

- ex:Movie1: Valid (Rating = 8.8).
- ex:Movie2: Valid (Rating = 9.5).
- ex:Movie3: Invalid (Rating = 12.0 is outside the range [0, 10]).

Example: SPARQL Constraint (2/2)

RDF Graph:

```
ex:Movie1 a ex:MOVIE ;
    ex:title "Inception" ;
    ex:year "2010" ;
    ex:awardYear "2010" .

ex:Movie2 a ex:MOVIE ;
    ex:title "The Matrix" ;
    ex:year "1999" ;
    ex:awardYear "2000" .

ex:Movie3 a ex:MOVIE ;
    ex:title "Avatar" ;
    ex:year "2009" ;
    ex:awardYear "2009" .
```

SHACL Shapes:

```
ex:AwardedMovieShape a sh:NodeShape ;
    sh:targetClass ex:MOVIE ;
    sh:sparql [
        a sh:SPARQLConstraint ;
        sh:message "Award year must match release year"
        sh:select ""
        SELECT $this
        WHERE {$this ex:year ?year ;
            ex:awardYear ?awardYear .
            FILTER (?year != ?awardYear)}"" ] .
```

Evaluation Results:

- ex:Movie1: Valid (Release year = 2010, Award year = 2010).
- ex:Movie2: Invalid (Release year = 1999, Award year = 2000).
- ex:Movie3: Valid (Release year = 2009, Award year = 2009).

SHACL: Boolean Constraints

- SHACL provides logical operators to express boolean constraints over shapes:
 - `sh:not SP`: Specifies that the target node must **not** conform to shape *SP*.
 - `sh:or (SP1 SP2 ... SPn)`: At least one of the shapes *SP1*, *SP2*, ..., *SPn* must be satisfied (logical disjunction).
 - `sh:and (SP1 SP2 ... SPn)`: All shapes *SP1*, *SP2*, ..., *SPn* must be satisfied (logical conjunction).
 - `sh:xone (SP1 SP2 ... SPn)`: Exactly one of the shapes *SP1*, *SP2*, ..., *SPn* must be satisfied (exclusive disjunction).
- Boolean constraints allow for complex validation rules combining multiple shapes.

Example: SHACL Boolean Constraints

RDF Graph:

```
ex:Movie1 a ex:MOVIE ;
    ex:title "Inception" ;
    ex:genre "Sci-Fi" .
ex:Movie2 a ex:MOVIE ;
    ex:title "The Matrix" ;
    ex:genre "Action" .
ex:Movie3 a ex:MOVIE ;
    ex:title "Documentary Example" ;
    ex:genre "Documentary" .
```

SHACL Shapes:

```
ex:NonDocumentaryShape a sh:NodeShape ;
    sh:not [
        sh:property [
            sh:path ex:genre ;
            sh:hasValue "Documentary" ] ] .
ex:SciFiOrActionShape a sh:NodeShape ;
    sh:or (
        [ sh:property [sh:path ex:genre ;
            sh:hasValue "Sci-Fi"]]
        [ sh:property [sh:path ex:genre ;
            sh:hasValue "Action"]]) .
```

Example: SHACL Boolean Constraints

RDF Graph:

```
ex:Movie1 a ex:MOVIE ;
    ex:title "Inception" ;
    ex:genre "Sci-Fi" .
ex:Movie2 a ex:MOVIE ;
    ex:title "The Matrix" ;
    ex:genre "Action" .
ex:Movie3 a ex:MOVIE ;
    ex:title "Documentary Example" ;
    ex:genre "Documentary" .
```

SHACL Shapes:

```
ex:NonDocumentaryShape a sh:NodeShape ;
    sh:not [
        sh:property [
            sh:path ex:genre ;
            sh:hasValue "Documentary" ] ] .
ex:SciFiOrActionShape a sh:NodeShape ;
    sh:or (
        [ sh:property [sh:path ex:genre ;
            sh:hasValue "Sci-Fi"]]
        [ sh:property [sh:path ex:genre ;
            sh:hasValue "Action"]]) .
```

Evaluation Results:

- ex:Movie1:
 - Valid for ex:SciFiOrActionShape ("Sci-Fi" matches).
 - Valid for ex:NonDocumentaryShape.
- ex:Movie2:
 - Valid for ex:SciFiOrActionShape ("Action" matches).
 - Valid for ex:NonDocumentaryShape.
- ex:Movie3:
 - Invalid for ex:NonDocumentaryShape ("Documentary" found).
 - Invalid for ex:SciFiOrActionShape.

SHACL: Value Type Constraints

- SHACL allows specifying constraints on the value types of target nodes:
 - `sh:class C`: Nodes must belong to class *C* (as defined in the RDF graph).
 - `sh:datatype D`: Nodes must have the datatype *D* (e.g., `xsd:string`, `xsd:integer`).
 - `sh:nodeKind K`: Nodes must be of a specific kind *K*, where:
 - `sh:IRI`: Node must be an IRI.
 - `sh:BlankNode`: Node must be a blank node.
 - `sh:Literal`: Node must be a literal.
 - `sh:BlankNodeOrIRI`: Node can be a blank node or an IRI.
 - `sh:BlankNodeOrLiteral`: Node can be a blank node or a literal.
 - `sh:IRIOrLiteral`: Node can be an IRI or a literal.
- These constraints ensure that the type and structure of target nodes conform to the expected values.

Example: SHACL Value Type Constraints

RDF Graph:

```
ex:Movie1 a ex:MOVIE ;
    ex:title "Inception" ;
    ex:year "2010"^^xsd:string ;
    ex:director ex:ChristopherNolan .
ex:Movie2 a ex:MOVIE ;
    ex:title "The Matrix" ;
    ex:year 1999 ;
    ex:director _:b1 .
ex:Movie3 a ex:MOVIE ;
    ex:title "Generic Movie" ;
    ex:year "Invalid Year" ;
    ex:director "Some Director" .
```

SHACL Shapes:

```
ex:MovieShape a sh:NodeShape ;
    sh:targetClass ex:MOVIE ;
    sh:property [
        sh:path ex:year ;
        sh:datatype xsd:integer ] ;
    sh:property [
        sh:path ex:director ;
        sh:nodeKind sh:BlankNodeOrIRI ] .
```

Example: SHACL Value Type Constraints

RDF Graph:

```
ex:Movie1 a ex:MOVIE ;
    ex:title "Inception" ;
    ex:year "2010"^^xsd:string ;
    ex:director ex:ChristopherNolan .
ex:Movie2 a ex:MOVIE ;
    ex:title "The Matrix" ;
    ex:year 1999 ;
    ex:director _:b1 .
ex:Movie3 a ex:MOVIE ;
    ex:title "Generic Movie" ;
    ex:year "Invalid Year" ;
    ex:director "Some Director" .
```

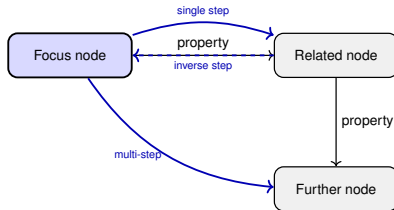
SHACL Shapes:

```
ex:MovieShape a sh:NodeShape ;
    sh:targetClass ex:MOVIE ;
    sh:property [
        sh:path ex:year ;
        sh:datatype xsd:integer ] ;
    sh:property [
        sh:path ex:director ;
        sh:nodeKind sh:BlankNodeOrIRI ] .
```

Evaluation Results:

- ex:Movie1:
 - Invalid: ex:year has datatype xsd:string instead of xsd:integer.
 - Valid: ex:director is an IRI (ex:ChristopherNolan).
- ex:Movie2:
 - Valid: ex:year is an xsd:integer.
 - Valid: ex:director is a blank node (_:b1), satisfying sh:BlankNodeOrIRI.
- ex:Movie3:
 - Invalid: ex:year has an invalid datatype.
 - Invalid: ex:director is a literal but expected to be sh:BlankNodeOrIRI.

SHACL Paths — An Intuition



Intuition

A SHACL path defines *how to navigate the graph* starting from a focus node, in order to reach values to be checked.

SHACL: Paths

- The property `sh:path` allows defining paths in SHACL shapes for navigating RDF graphs.
- Common path constructs include:
 - `ex:p`: Represents a direct property path `ex:p`.
 - `[sh:inversePath e]`: Represents the inverse of path `e`.
 - `(e f)`: Represents a sequence of paths `e` followed by `f`.
 - `[sh:alternativePath (e f)]`: Represents a choice between paths `e` or `f`.
 - `[sh:zeroOrMorePath e]`: Represents recursive `e` (zero or more occurrences).
 - `[sh:oneOrMorePath e]`: Represents recursive `e` (at least one occurrence).
 - `[sh:zeroOrOnePath e]`: Represents an optional `e`.

Example: SHACL `sh:inversePath`

RDF Graph:

```
ex:Movie1 a ex:MOVIE ;
    ex:title "Inception" ;
    ex:hasDirector ex:Director1 .

ex:Director1 a ex:DIRECTOR ;
    ex:name "Christopher Nolan" .
```

SHACL Shapes:

```
ex:DirectorShape a sh:NodeShape ;
    sh:targetClass ex:DIRECTOR ;
    sh:property [
        sh:path [sh:inversePath ex:hasDirector] ;
        sh:class ex:MOVIE ;
        sh:minCount 1 ;
    ] .
```

Example: SHACL `sh:inversePath`

RDF Graph:

```
ex:Movie1 a ex:MOVIE ;  
    ex:title "Inception" ;  
    ex:hasDirector ex:Director1 .  
  
ex:Director1 a ex:DIRECTOR ;  
    ex:name "Christopher Nolan" .
```

SHACL Shapes:

```
ex:DirectorShape a sh:NodeShape ;  
    sh:targetClass ex:DIRECTOR ;  
    sh:property [  
        sh:path [sh:inversePath ex:hasDirector] ;  
        sh:class ex:MOVIE ;  
        sh:minCount 1 ;  
    ] .
```

Explanation:

- `sh:inversePath ex:hasDirector` checks for movies (`ex:MOVIE`) that are directed by the current node.

Evaluation Results:

- `ex:Director1`: Valid. It is the director of `ex:Movie1`.

Example: SHACL Path Sequence (e f) (1/2)

RDF Graph:

```
ex:Movie1 a ex:MOVIE ;  
    ex:hasCharacter ex:Character1 .  
  
ex:Character1 a ex:CHARACTER ;  
    ex:hasActor ex:Actor1 .  
  
ex:Actor1 a ex:ACTOR ;  
    ex:name "Leonardo DiCaprio" .
```

SHACL Shapes:

```
ex:MovieActorShape a sh:NodeShape ;  
    sh:targetClass ex:MOVIE ;  
    sh:property [  
        sh:path (ex:hasCharacter ex:hasActor) ;  
        sh:class ex:ACTOR ;  
        sh:minCount 1 ;  
    ] .
```

Explanation:

- The path (ex:hasCharacter ex:hasActor) navigates from movies to their actors via characters.

Evaluation Results:

- ex:Movie1: Valid. It connects to ex:Actor1 via ex:Character1.

Example: SHACL Sequence Path (e f) (2/2)

RDF Graph:

```
ex:Movie1 a ex:MOVIE ;  
    ex:title "Inception" ;  
    ex:hasDirector ex:Director1 .  
  
ex:Director1 a ex:DIRECTOR ;  
    ex:worksFor ex:Studio1 .  
  
ex:Studio1 a ex:STUDIO ;  
    ex:name "Warner Bros" .
```

SHACL Shapes:

```
ex:StudioShape a sh:NodeShape ;  
    sh:property [  
        sh:path (ex:hasDirector ex:worksFor) ;  
        sh:nodeKind sh:IRI ;  
    ] .
```

Evaluation Results:

- ex:Movie1:
 - Valid: Path (ex:hasDirector ex:worksFor) connects ex:Movie1 to ex:Studio1.
- ex:Director1 and ex:Studio1 are not evaluated directly.

SHACL: Closed Constraints and Types of Shapes

- SHACL provides mechanisms to restrict additional properties in target nodes:
 - `sh:closed true:`
 - Only properties explicitly specified in the shape are allowed.
 - Properties not enumerated as values of `sh:path` result in validation errors.
 - `sh:ignoredProperties:`
 - Specifies a list of additional properties that are permitted despite the `sh:closed` rule.

Types of Shapes:

- **Open Shapes:** Target nodes can have additional properties beyond those specified in the shape.
- **Closed Shapes:** Target nodes are **not** allowed to have properties other than those explicitly specified in the shape.

Example: SHACL Closed Constraints

RDF Graph:

```
ex:Dose1 a ex:PrescribedDose ;
    ex:dose "50mg" ;
    ex:start_date "2023-01-01"^^xsd:date ;
    ex:end_date "2023-01-10"^^xsd:date ;
    ex:notes "Take with food" .

ex:Dose2 a ex:PrescribedDose ;
    ex:dose "100mg" ;
    ex:start_date "2023-01-15"^^xsd:date ;
    ex:end_date "2023-01-20"^^xsd:date .
```

SHACL Shape:

```
ex:PrescribedDoseShape a sh:NodeShape ;
    sh:targetClass ex:PrescribedDose ;
    sh:closed true ;
    sh:property [
        sh:path ex:dose ;
        sh:datatype xsd:string ;
        sh:minCount 1 ; sh:maxCount 1 ] ;
    sh:property [
        sh:path ex:start_date ;
        sh:datatype xsd:date ;
        sh:minCount 1 ; sh:maxCount 1 ] ;
    sh:property [
        sh:path ex:end_date ;
        sh:datatype xsd:date ;
        sh:minCount 1 ; sh:maxCount 1 ] .
```

Example: SHACL Closed Constraints

RDF Graph:

```
ex:Dose1 a ex:PrescribedDose ;
    ex:dose "50mg" ;
    ex:start_date "2023-01-01"^^xsd:date ;
    ex:end_date "2023-01-10"^^xsd:date ;
    ex:notes "Take with food" .

ex:Dose2 a ex:PrescribedDose ;
    ex:dose "100mg" ;
    ex:start_date "2023-01-15"^^xsd:date ;
    ex:end_date "2023-01-20"^^xsd:date .
```

SHACL Shape:

```
ex:PrescribedDoseShape a sh:NodeShape ;
    sh:targetClass ex:PrescribedDose ;
    sh:closed true ;
    sh:property [
        sh:path ex:dose ;
        sh:datatype xsd:string ;
        sh:minCount 1 ; sh:maxCount 1 ] ;
    sh:property [
        sh:path ex:start_date ;
        sh:datatype xsd:date ;
        sh:minCount 1 ; sh:maxCount 1 ] ;
    sh:property [
        sh:path ex:end_date ;
        sh:datatype xsd:date ;
        sh:minCount 1 ; sh:maxCount 1 ] .
```

Explanation:

- `sh:closed true` ensures only `ex:dose`, `ex:start_date`, and `ex:end_date` are allowed.

Evaluation Results:

- `ex:Dose1`: Invalid. Property `ex:notes` is not permitted.
- `ex:Dose2`: Valid. Only the allowed properties are specified.

SHACL - Summary (1/2)

Type Constraint	SHACL Predicate
Cardinality	sh:minCount m: there are at least m nodes connected with each target node via a path p sh:maxCount m: there are at most m nodes connected with each target node via a path p sh:qualifiedMinCount m T: at least m nodes connected via p satisfy shapes T sh:qualifiedMaxCount m T: at most m nodes connected via p satisfy shapes T
Target Nodes	sh:targetClass C: target nodes are the entities in class C sh:targetSubjectsOf P: target nodes are the subjects of P sh:targetObjectsOf P: target nodes are the objects of P
Value Range Constraints	sh:minInclusive m: $n \geq m$ sh:maxInclusive m: $n \leq m$ sh:minExclusive m: $n > m$ sh:maxExclusive m: $n < m$ sh:hasValue v: v belongs to the set of target nodes N sh:in (v1...vn): N is a subset of {v1, ..., vn}
SPARQL Constraints	sh:sparql: declares a constraint specified as a SPARQL query sh:SPARQLConstraint: class of SPARQL query constraints sh:message: human-readable message explaining the cause of the violation sh:select: declares the SPARQL query. Variable \$this is bound to target nodes

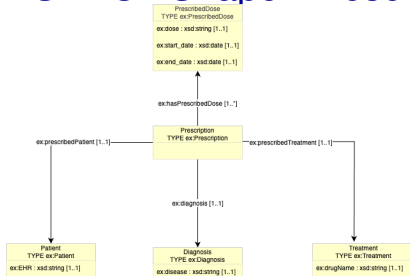
SHACL - Summary (2/2)

Type Constraint	SHACL Predicate
Boolean Constraints	sh:not SP: negation of the shape SP sh:or (SP1 SP2 ... SPn): disjunction of SP1 SP2 ... SPn sh:and (SP1 SP2 ... SPn): conjunction of SP1 SP2 ... SPn sh:xone (SP1 SP2 ... SPn): exclusive disjunction of SP1 SP2 ... SPn
Value Type Constraints	sh:class C: target nodes must belong to class C sh:datatype D: target nodes must have the datatype D sh:nodeKind K: target nodes must be of the kind K, i.e., sh:IRI, sh:BlankNode, sh:Literal, sh:BlankNodeOrIRI, sh:BlankNodeOrLiteral, sh:IRIOrLiteral
Paths (sh:path)	P: the path corresponds to the property P [sh:inversePath e]: inverse of e (e f): e then f [sh:alternativePath (e f)]: e or f [sh:zeroOrMorePath e]: recursive e [sh:oneOrMorePath e]: recursive e with at least one [sh:zeroOrOnePath e]: optional e
Close Constraints	sh:closed true: only properties in shapes graph are allowed. sh:ignoredProperties: specifies a list of properties that are permitted in addition to those enumerated by sh:path.

Exercise

Participate **in the survey at Stud.IP.**

SHACL Shape: Prescription

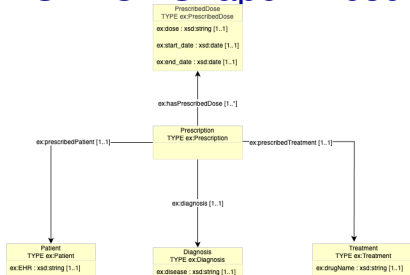


SHACL Shape: Prescription

```

ex:PrescriptionShape a sh:NodeShape ;
  sh:targetClass ex:Prescription ;
  sh:property [
    sh:path ex:prescribedPatient ;
    sh:class ex:Patient ;
    sh:minCount 1 ; sh:maxCount 1 ] ;
  sh:property [
    sh:path ex:diagnosis ;
    sh:class ex:Diagnosis ;
    sh:minCount 1 ; sh:maxCount 1 ] ;
  sh:property [
    sh:path ex:hasPrescribedDose ;
    sh:class ex:PrescribedDose ] ;
  sh:property [
    sh:path ex:prescribedTreatment ;
    sh:class ex:Treatment ;
    sh:minCount 1 ; sh:maxCount 1 ] .
  
```

SHACL Shape: Prescription



SHACL Shape: Prescription

```

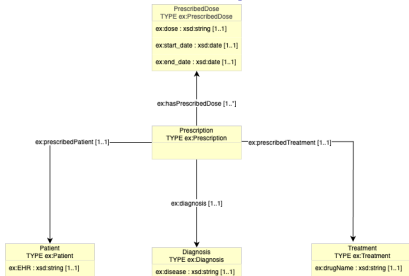
ex:PrescriptionShape a sh:NodeShape ;
  sh:targetClass ex:Prescription ;
  sh:property [
    sh:path ex:prescribedPatient ;
    sh:class ex:Patient ;
    sh:minCount 1 ; sh:maxCount 1 ] ;
  sh:property [
    sh:path ex:diagnosis ;
    sh:class ex:Diagnosis ;
    sh:minCount 1 ; sh:maxCount 1 ] ;
  sh:property [
    sh:path ex:hasPrescribedDose ;
    sh:class ex:PrescribedDose ] ;
  sh:property [
    sh:path ex:prescribedTreatment ;
    sh:class ex:Treatment ;
    sh:minCount 1 ; sh:maxCount 1 ] .

```

Observations:

- A Prescription must be linked to exactly one patient (ex:prescribedPatient) and one diagnosis.
- A Prescription must include at least one treatment and optionally one or more prescribed doses.

SHACL Shape: Prescribed Dose

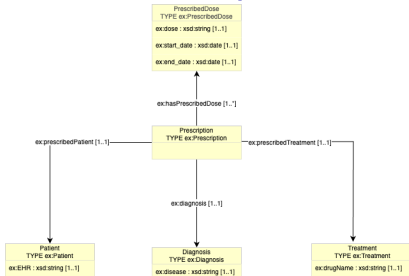


SHACL Shape: Prescribed Dose

```

ex:PrescribedDoseShape a sh:NodeShape ;
  sh:targetClass ex:PrescribedDose ;
  sh:property [
    sh:path ex:dose ;
    sh:datatype xsd:string ;
    sh:minCount 1 ; sh:maxCount 1 ] ;
  sh:property [
    sh:path ex:start_date ;
    sh:datatype xsd:date ;
    sh:minCount 1 ; sh:maxCount 1 ] ;
  sh:property [
    sh:path ex:end_date ;
    sh:datatype xsd:date ;
    sh:minCount 1 ; sh:maxCount 1 ] .
  
```

SHACL Shape: Prescribed Dose



SHACL Shape: Prescribed Dose

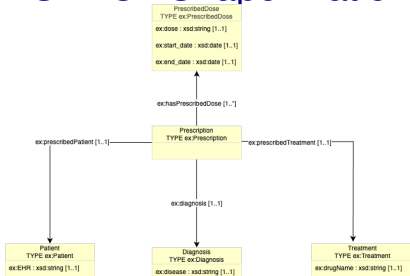
```

ex:PrescribedDoseShape a sh:NodeShape ;
  sh:targetClass ex:PrescribedDose ;
  sh:property [
    sh:path ex:dose ;
    sh:datatype xsd:string ;
    sh:minCount 1 ; sh:maxCount 1 ] ;
  sh:property [
    sh:path ex:start_date ;
    sh:datatype xsd:date ;
    sh:minCount 1 ; sh:maxCount 1 ] ;
  sh:property [
    sh:path ex:end_date ;
    sh:datatype xsd:date ;
    sh:minCount 1 ; sh:maxCount 1 ] .
  
```

Observations:

- A PrescribedDose must have a dose (string) and two mandatory dates (start_date and end_date).

SHACL Shape: Patient

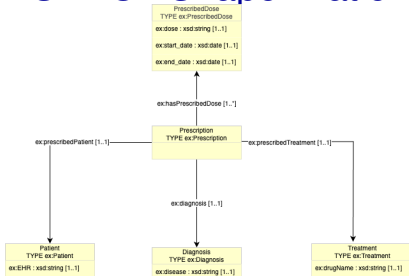


SHACL Shape: Patient

```

ex:PatientShape a sh:NodeShape ;
  sh:targetClass ex:Patient ;
  sh:property [
    sh:path ex:EHR ;
    sh:datatype xsd:string ;
    sh:minCount 1 ; sh:maxCount 1 ] .
  
```


SHACL Shape: Patient



SHACL Shape: Patient

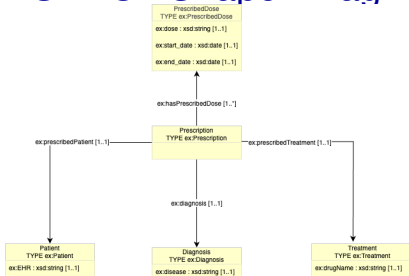
```

ex:PatientShape a sh:NodeShape ;
  sh:targetClass ex:Patient ;
  sh:property [
    sh:path ex:EHR ;
    sh:datatype xsd:string ;
    sh:minCount 1 ; sh:maxCount 1 ] .
  
```

Observations:

- A Patient must have an EHR (Electronic Health Record), represented as a string.

SHACL Shape: Diagnosis

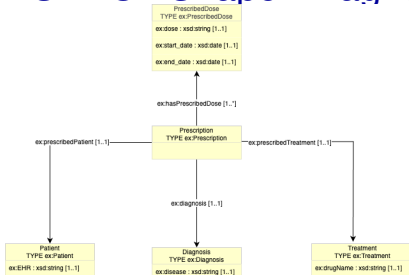


SHACL Shape: Diagnosis

```

ex:DiagnosisShape a sh:NodeShape ;
  sh:targetClass ex:Diagnosis ;
  sh:property [
    sh:path ex:disease ;
    sh:datatype xsd:string ;
    sh:minCount 1 ; sh:maxCount 1 ] .
  
```

SHACL Shape: Diagnosis



SHACL Shape: Diagnosis

```
ex:DiagnosisShape a sh:NodeShape ;
  sh:targetClass ex:Diagnosis ;
  sh:property [
    sh:path ex:disease ;
    sh:datatype xsd:string ;
    sh:minCount 1 ; sh:maxCount 1 ] .
```

Observations:

- A Diagnosis must include the disease name (`ex:disease`), represented as a string.

Example: Qualified Cardinality Constraints

RDF Graph:

```
ex:Prescription1 a ex:Prescription ;
  ex:hasPrescribedDose ex:Dose1 ;
  ex:hasPrescribedDose ex:Dose2 ;
  ex:hasPrescribedDose ex:Dose3 .
ex:Dose1 a ex:PrescribedDose ;
  ex:dose "50mg" ;
  ex:start_date "2023-01-01"^^xsd:date .
ex:Dose2 a ex:PrescribedDose ;
  ex:dose "100mg" ;
  ex:start_date "2023-01-15"^^xsd:date .
ex:Dose3 a ex:OtherDose ;
  ex:dose "75mg" .
```

SHACL Shape:

```
ex:PrescriptionShape a sh:NodeShape ;
  sh:targetClass ex:Prescription ;
  sh:property [
    sh:path ex:hasPrescribedDose ;
    sh:qualifiedMinCount 2 ;
    sh:qualifiedMaxCount 2 ;
    sh:qualifiedValueShape [
      sh:class ex:PrescribedDose ;
    ] ;
  ] .
```

Explanation:

- `sh:qualifiedMinCount 2` and `sh:qualifiedMaxCount 2` ensure exactly two nodes satisfy the shape.
- The qualified value shape checks that the nodes belong to the class `ex:PrescribedDose`.

Evaluation Results:

- `ex:Prescription1`:
 - `ex:Dose1` and `ex:Dose2` satisfy the shape (`ex:PrescribedDose`).
 - `ex:Dose3` does not satisfy the shape (`ex:OtherDose`).
- `ex:Prescription1`: Valid. Exactly two doses satisfy the qualified constraint.

Example: SHACL with Inverse Path

RDF Graph:

```
ex:Patient1 a ex:Patient ;  
  ex:EHR "EHR-001" .  
  
ex:Prescription1 a ex:Prescription ;  
  ex:prescribedPatient ex:Patient1 .  
  
ex:Prescription2 a ex:Prescription ;  
  ex:prescribedPatient ex:Patient1 .
```

SHACL Shape:

```
ex:PatientShape a sh:NodeShape ;  
  sh:targetClass ex:Patient ;  
  sh:property [  
    sh:path [  
      sh:inversePath ex:prescribedPatient] ;  
      sh:class ex:Prescription ;  
      sh:minCount 1 ; sh:maxCount 2  
    ] .
```

Explanation:

- `sh:inversePath ex:prescribedPatient` navigates in the reverse direction of the property `ex:prescribedPatient`.
- The shape ensures that each `ex:Patient` node is linked to between 1 and 2 `ex:Prescription` nodes.

Evaluation Results:

- `ex:Patient1`:
 - Linked to `ex:Prescription1` and `ex:Prescription2` via the inverse of `ex:prescribedPatient`.
 - Satisfies the constraint: `minCount 1` and `maxCount 2`.

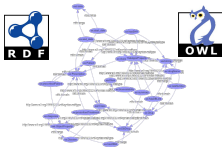
Constraints: SPARQL vs OWL vs SHACL

Aspect	SPARQL	OWL	SHACL
Primary role	Query language	Ontology & reasoning language	Constraint & validation language
How constraints are expressed	As query patterns detecting violations	As logical axioms and restrictions	As shapes with explicit constraints
Assumption about missing data	Closed World What is not found is considered false	Open World Missing data may exist elsewhere	Closed World Missing required data is a violation
Typical use of constraints	Error detection and quality checks	Inference and semantic enrichment	Validation and quality assessment
Validation outcome	Query returns results (violations)	No notion of validation failure	Explicit conformance / violation report
Best suited for	Ad-hoc checks and debugging	Knowledge modeling and reasoning	Systematic data quality validation

Key takeaway

- **OWL** follows the **Open World Assumption (OWA)**: absence of data does *not* imply an error.
- **SPARQL** and **SHACL** follow a **Closed World Assumption (CWA)**: missing expected data is treated as a *violation*.

Summary- Core Concepts and Insights



Ontology describes the universe of discourse and provides a unified view of the data sources,

ID	Name	Type	Size	Date	Status
1	Source 1	Database	100 MB	2023-01-01	Active
2	Source 2	File	50 MB	2023-01-02	Active
3	Source 3	API	10 MB	2023-01-03	Active
4	Source 4	Database	200 MB	2023-01-04	Active
5	Source 5	File	75 MB	2023-01-05	Active

Data sources that provide the data that will be integrated.

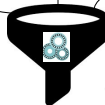
```

1 @prefix owl: <http://www.w3.org/2002/07/owl#> .
2 @prefix rdfs: <http://www.w3.org/2000/01/rdf-schema#> .
3 @prefix xsd: <http://www.w3.org/2001/XMLSchema#> .
4 @prefix sh: <http://www.w3.org/ns/shacl#> .
5 @prefix ex: <http://www.semanticweb.org/example/ontology#> .
6
7 # Example SHACL rules
8
9 # Rule 1: ex:Person must be an instance of ex:Human
10 sh:PropertyPath ex:hasChild sh:hasType ex:Human
11
12 # Rule 2: ex:Person must have a unique value for ex:name
13 sh:PropertyPath ex:name sh:unique
14
15 # Rule 3: ex:Person must have a value for ex:age
16 sh:PropertyPath ex:age sh:minValue 0
17
18 # Rule 4: ex:Person must have a value for ex:email
19 sh:PropertyPath ex:email sh:minLength 10
20
21 # Rule 5: ex:Person must have a value for ex:phone
22 sh:PropertyPath ex:phone sh:minLength 10
23
24 # Rule 6: ex:Person must have a value for ex:address
25 sh:PropertyPath ex:address sh:minLength 10
26
27 # Rule 7: ex:Person must have a value for ex:city
28 sh:PropertyPath ex:city sh:minLength 10
29
30 # Rule 8: ex:Person must have a value for ex:country
31 sh:PropertyPath ex:country sh:minLength 10
32
33 # Rule 9: ex:Person must have a value for ex:gender
34 sh:PropertyPath ex:gender sh:minLength 10
35
36 # Rule 10: ex:Person must have a value for ex:maritalStatus
37 sh:PropertyPath ex:maritalStatus sh:minLength 10
38
39 # Rule 11: ex:Person must have a value for ex:occupation
40 sh:PropertyPath ex:occupation sh:minLength 10
41
42 # Rule 12: ex:Person must have a value for ex:education
43 sh:PropertyPath ex:education sh:minLength 10
44
45 # Rule 13: ex:Person must have a value for ex:experience
46 sh:PropertyPath ex:experience sh:minLength 10
47
48 # Rule 14: ex:Person must have a value for ex:skills
49 sh:PropertyPath ex:skills sh:minLength 10
50
51 # Rule 15: ex:Person must have a value for ex:interests
52 sh:PropertyPath ex:interests sh:minLength 10
53
54 # Rule 16: ex:Person must have a value for ex:hobbies
55 sh:PropertyPath ex:hobbies sh:minLength 10
56
57 # Rule 17: ex:Person must have a value for ex:pets
58 sh:PropertyPath ex:pets sh:minLength 10
59
60 # Rule 18: ex:Person must have a value for ex:vehicles
61 sh:PropertyPath ex:vehicles sh:minLength 10
62
63 # Rule 19: ex:Person must have a value for ex:travelHistory
64 sh:PropertyPath ex:travelHistory sh:minLength 10
65
66 # Rule 20: ex:Person must have a value for ex:favoriteFood
67 sh:PropertyPath ex:favoriteFood sh:minLength 10
68
69 # Rule 21: ex:Person must have a value for ex:favoriteColor
70 sh:PropertyPath ex:favoriteColor sh:minLength 10
71
72 # Rule 22: ex:Person must have a value for ex:favoriteMusic
73 sh:PropertyPath ex:favoriteMusic sh:minLength 10
74
75 # Rule 23: ex:Person must have a value for ex:favoriteMovie
76 sh:PropertyPath ex:favoriteMovie sh:minLength 10
77
78 # Rule 24: ex:Person must have a value for ex:favoriteBook
79 sh:PropertyPath ex:favoriteBook sh:minLength 10
80
81 # Rule 25: ex:Person must have a value for ex:favoriteTVShow
82 sh:PropertyPath ex:favoriteTVShow sh:minLength 10
83
84 # Rule 26: ex:Person must have a value for ex:favoriteVideoGame
85 sh:PropertyPath ex:favoriteVideoGame sh:minLength 10
86
87 # Rule 27: ex:Person must have a value for ex:favoriteSport
88 sh:PropertyPath ex:favoriteSport sh:minLength 10
89
90 # Rule 28: ex:Person must have a value for ex:favoriteTeam
91 sh:PropertyPath ex:favoriteTeam sh:minLength 10
92
93 # Rule 29: ex:Person must have a value for ex:favoriteArtist
94 sh:PropertyPath ex:favoriteArtist sh:minLength 10
95
96 # Rule 30: ex:Person must have a value for ex:favoriteBand
97 sh:PropertyPath ex:favoriteBand sh:minLength 10
98
99 # Rule 31: ex:Person must have a value for ex:favoriteGenre
100 sh:PropertyPath ex:favoriteGenre sh:minLength 10
  
```

Mapping Rules establish the correspondences between the ontology and the data sources.

1

The execution of the mapping rules on the data sources creates a knowledge graph.



Mapping-rule engine

SHACL engine



2

Quality of the data integrated in the knowledge graph is validated using integrity constraints.



3

Queries express graph patterns to be evaluated over the knowledge graph.



The **knowledge graph** comprises the instances of the classes and properties in the ontology; it also includes the ontology.



References I



A. Hogan, E. Blomqvist, M. Cochez, C. d'Amato, G. de Melo, C. Gutierrez, S. Kirrane, J. E. L. Gayo, R. Navigli, S. Neumaier, A. N. Ngomo, A. Polleres, S. M. Rashid, A. Rula, L. Schmelzeisen, J. F. Sequeda, S. Staab, and A. Zimmermann. Knowledge graphs. *ACM Comput. Surv.*, 54(4):71:1–71:37, 2022.